

Ministerul Educației al Republicii Moldova Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și microelectronică
Departamentul Inginerie Biomedicală

Raport

La programarea calculatoarelor

Tema: Introducere în Limbaj de programare

Efectuat de st.gr. IBM-251, Gurdiș Roman
Verificat, asistentă universitară: Rotari Augustina

Chișinău 2025

Lucrare practică nr. 2 la disciplina programarea calculatoarelor

Scopul lucrării

Scopul lucrării este de a familiariza studentul cu instrucțiunile de ramificare și ciclice a limbajului C.

Rezultatul lucrării:

La finele lucrării studentul trebuie să cunoască:

- ce instrucțiuni de ramificare sunt în limbajul C?
- ce instrucțiuni ciclice sunt în limbajul C?
- care este diferența între while și do while?
- care este diferența între for și while?
- care este diferența între if și switch?
- ce înseamnă break, continue ?
- ce înseamnă contor, condiție de oprire și pasul contorului?

Probleme pentru instrucțiuni de ramificare:

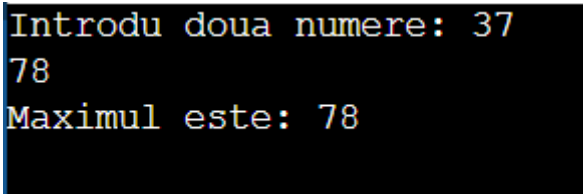
Exercițiu nr.1

1. Scrieți un program C pentru a găsi maxim între două numere.

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Introdu doua numere: ");
    scanf("%d %d", &a, &b);

    if(a > b) printf("Maximul este: %d\n", a);
    else printf("Maximul este: %d\n", b);

    return 0;}
```



```
Introdu doua numere: 37
78
Maximul este: 78
```

„Figura 1. Rezultatul executării programului pentru valorile 37 și 78.”

Concluzie

„Programul arată corect numărul mai mare dintre două numere.

Probleme pentru instrucțiuni de ramificare:

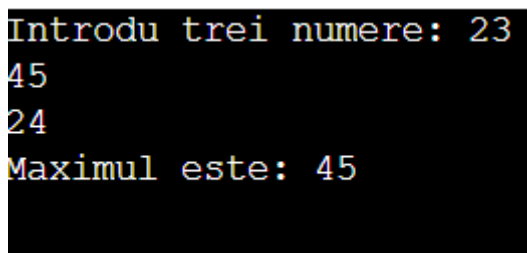
Exercițiu nr.2

2.Scrieți un program C pentru a găsi maxim între trei numere.

```
#include <stdio.h>
int main() {
    int a, b, c;
    printf("Introdu trei numere: ");
    scanf("%d %d %d", &a, &b, &c);

    int max = a;
    if(b > max) max = b;
    if(c > max) max = c;

    printf("Maximul este: %d\n", max);
    return 0;}
```



```
Introdu trei numere: 23
45
24
Maximul este: 45
```

„Figura 2. Rezultatul executării programului pentru valorile 23 45 23”

Concluzie

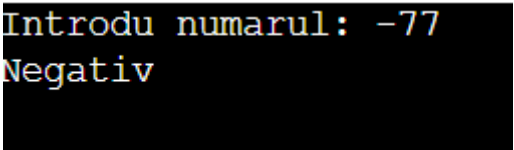
„Programul arată corect numărul mai mare dintre trei numere.”

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.3

3.Scrieți un program C pentru a verifica dacă un număr este negativ, pozitiv sau zero.

```
#include <stdio.h>
int main() {
    int n;
    printf("Introdu numarul: ");
    scanf("%d", &n);
    if(n > 0) printf("Pozitiv\n");
    else if(n < 0) printf("Negativ\n");
    else printf("Zero\n");
    return 0;}
```



```
Introdu numarul: -77
Negativ
```

“Figura 3. Rezultatele programului pentru verificarea semnului unui număr”

Concluzie

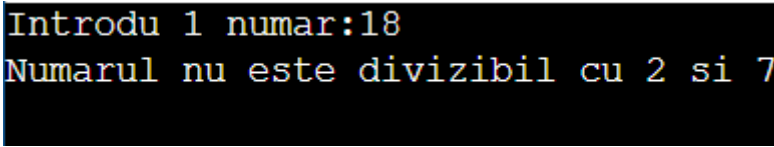
Programul afișează corect dacă un număr este pozitiv, negativ sau zero. Funcționează pentru orice număr introdus.

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.4

4.Scrieți un program C pentru a verifica dacă un număr este divizibil cu 5 și 11 sau nu.

```
#include<stdio.h>
#include<math.h>
int main() {
int num;
printf("Introdu 1 numar:");
scanf("%d", &num);
if (num % 2 == 0 && num % 7 == 0){
printf("Numarul este divizibil cu 2 si 7\n");
} else
{printf("Numarul nu este divizibil cu 2 si 7\n");
}
return 0;}
```



```
Introdu 1 numar:18
Numarul nu este divizibil cu 2 si 7
```

“Figura 4. Rezultatul unui numar introdus ”

Concluzie

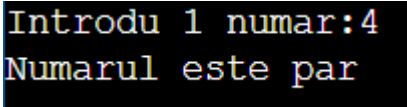
„Programul arată dacă numărul dat se împarte exact la 2 și 7.”

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.5

5.Scrieți un program C pentru a verifica dacă un număr este par sau impar.

```
#include<stdio.h>
#include<math.h>
int main() {
    int num;
    printf("Introdu 1 numar:");
    scanf("%d", &num);
    if (num % 2 == 0 ){
        printf("Numarul este par\n");
    } else
    {printf("Numarul este impar\n");
    }
    return 0;}
```



“Figura 4. “

Concluzie

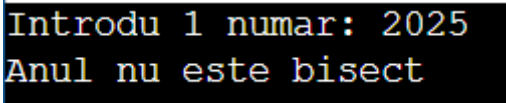
În această programă analizăm dacă numărul este par sau impar, în caz dacă numărul se împarte la 2, numărul este par dar dacă rămâne rest atunci el este impar.

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.6

6.Scrieți un program C pentru a verifica dacă un an este bisect sau nu.

```
#include<stdio.h>
#include<math.h>
int main() {
int num;
printf("Introdu 1 numar:");
scanf("%d", &num);
if (num % 4 == 0 ){
printf("Anul bisect\n");
} else
{printf("Anul nu este bisect\n");
}
return 0;}
```



```
Introdu 1 numar: 2025
Anul nu este bisect
```

"Figura 6."

Concluzie

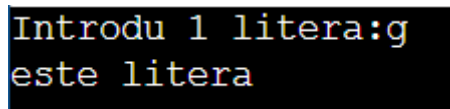
Acest cod arata daca anul este bisector sau nu

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.7

7.Scrieți un program C pentru a verifica dacă un caracter este sau nu alfabet.

```
#include<stdio.h>
#include<ctype.h>
int main() {
    char litera;
    printf("Introdu 1 litera:");
    scanf("%c", &litera);
    if (isalpha(litera) )
        printf("este litera\n");
    else
        printf("Nu este litera\n");
    return 0;}
```



```
Introdu 1 litera:g
este litera
```

“Figura 7.”

Concluzie

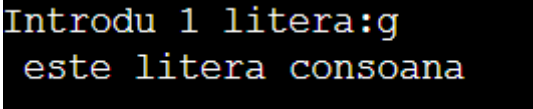
Codul verifica daca simbolul introdus este litera sau nu

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.8

8.Scrieți un program C pentru a introduce orice alfabet și verificați dacă este vocală sau consoană.

```
#include<stdio.h>
int main() {
    char litera;
    printf("Introdu 1 litera:");
    scanf("%c", &litera );
    if (litera == 'a' || litera == 'e' || litera ==
        'i' || litera == 'u' )
        printf("este litera vocala\n");
    else
        printf(" este litera consoana\n");
    return 0;}
```



```
Introdu 1 litera:g
este litera consoana
```

"Figura 8."

Concluzie

Codul afiseaza daca litera este vocala sau consoana

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.9

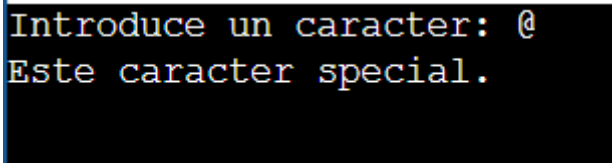
9.Scrieți un program C pentru a introduce orice caracter și verificați dacă este alfabet, cifră sau caracter special.

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char crt;

    printf("Introduce un caracter: ");
    scanf("%c", &crt);

    if (isalpha(crt)) {
        printf("Este litera alfabet.\n");
    } else if (isdigit(crt)) {
        printf("Este cifră.\n");
    } else {
        printf("Este caracter special.\n");
    }
    return 0;}
```



```
Introduce un caracter: @
Este caracter special.
```

"Figura 9."

Concluzie

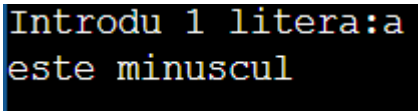
Codul afiseaza ce fel de caracter este introdus litera cifra sau un caracter special.

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.10

10.Scrieți un program C pentru a verifica dacă un caracter este cu majuscule sau minuscule.

```
#include<stdio.h>
#include<ctype.h>
int main() {
    char litera;
    printf("Introdu 1 litera:");
    scanf("%c", &litera);
    if (isalpha(litera) && (litera >= 'A' && litera <=
    'Z'))
    printf("este majuscul\n");
    else
    printf("este minuscul\n");
    return 0;}
```



```
Introdu 1 litera:a
este minuscul
```

“Figura 10. “

Concluzie

Codul afiseaza care din litere introduse sunt minuscule si care sunt majuscule.

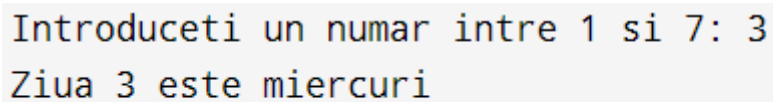
Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.11

11.Scrieți un program C pentru a introduce numărul săptămânii și a imprima ziua săptămânii.

```
#include <stdio.h>
int main() {
    int zi;
    char* zile[] = {"luni", "marti", "miercuri", "joi", "vineri", "sambata", "duminica"};
    printf("Introduceti un numar intre 1 si 7: ");
    scanf("%d", &zi);
    if (zi >= 1 && zi <= 7) {
        printf("Ziua %d este %s\n", zi, zile[zi - 1]);
    } else {
        printf("Numar invalid\n");
    }
    return 0;}

```



```
Introduceti un numar intre 1 si 7: 3
Ziua 3 este miercuri

```

“Figura 11.”

Concluzie

Codul print cifra respectiva arata ziua saptamanii

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.12

12.Scrieți un program C pentru a introduce numărul lunii și a imprima numărul de zile din luna respectivă.

```
#include <stdio.h>
```

```
int main() {  
    int luna;  
    int zile[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};  
  
    printf("Introduceti numarul lunii (1-12): ");  
    scanf("%d", &luna);  
  
    if (luna >= 1 && luna <= 12) {  
        printf("Luna %d are %d zile.\n", luna, zile[luna - 1]);  
    } else {  
        printf("Numar invalid.\n");  
    }  
    return 0;  
}
```

```
Introduceti numarul lunii (1-12): 7  
Luna 7 are 31 zile.
```

“Figura 12.”

Concluzie

Prin cifra data codul afiseaza luna care corespunde cifrei date si arata cate zile are aceasta luna

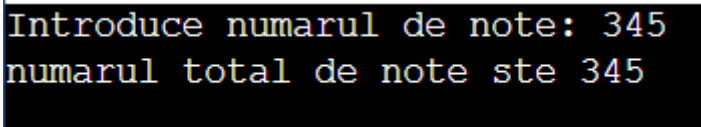
Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.13

13.Scrieți un program C pentru a număra numărul total de note într-o anumită cantitate

```
#include<stdio.h>

int main(){
    int n;
    printf("Introduce numarul de note: ");
    scanf("%d",&n);
    printf("numarul total de note ste %d\n",n);
    return 0; }
```



```
Introduce numarul de note: 345
numarul total de note ste 345
```

“Figura 13.”

Concluzie

Codul dat arata numarul total

Probleme pentru instrucțiuni de ramificare:

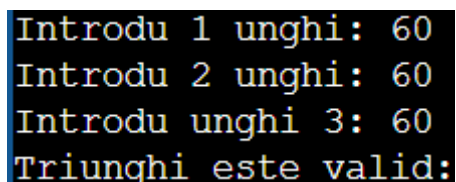
Exercițiu nr.14

14. Scrieți un program C pentru a introduce unghiurile unui triunghi și verificați dacă triunghiul este valid sau nu.

```
#include<stdio.h>

int main () {
    int a,b,c;

    printf("Introdu 1 unghi: ");
    scanf("%d", &a);
    printf("Introdu 2 unghi: ");
        scanf("%d", &b);
    printf("Introdu unghi 3: ");
        scanf("%d", &c);
    if(a+b+c==180)
        printf("Triunghi este valid: ");
    else
        printf("Triunghi nu este valid: ");
    return 0; }
```



```
Introdu 1 unghi: 60
Introdu 2 unghi: 60
Introdu unghi 3: 60
Triunghi este valid:
```

“Figura 14.”

Concluzie

Codul arata cate grade are fiecare unghi

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.15

15.Scrieți un program C pentru a introduce toate laturile unui triunghi și verificați dacă triunghiul este valid sau nu

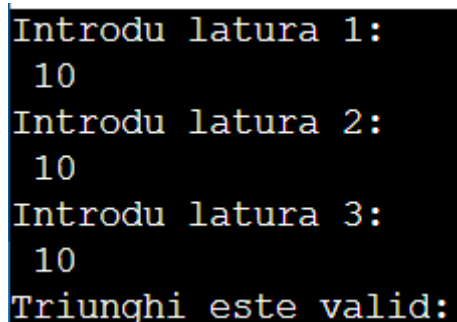
```
#include<stdio.h>

int main () {
int a,b,c;

printf("Introdu latura 1:\n ");
scanf("%d", &a);
printf("Introdu latura 2:\n ");
scanf("%d", &b);
printf("Introdu latura 3:\n ");
scanf("%d", &c);

if (a+b<=c || a+c<=b || b+c<=a)
printf("Triunghi nu este valid: ");
else
printf("Triunghi este valid: ");

return 0; }
```



```
Introdu latura 1:
10
Introdu latura 2:
10
Introdu latura 3:
10
Triunghi este valid:
```

“Figura 15.”

Concluzie

Programul citește cele trei laturi ale unui triunghi și verifică dacă suma oricăror două laturi este mai mare decât a treia;

Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.16

16.Scrieți un program C pentru a verifica dacă triunghiul este echilateral, isoscel sau triunghi scalen

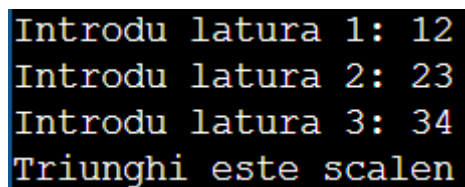
```
#include<stdio.h>

int main () {
int a,b,c;

printf("Introdu latura 1: ");
scanf("%d", &a);
printf("Introdu latura 2: ");
scanf("%d", &b);
printf("Introdu latura 3: ");
scanf("%d", &c);

if ( a <= 0 || b <= 0 || c <= 0 || a+b<=c || a+c<=b
|| b+c<=a)

printf("Triunghi nu este valid: ");
else if (a == b && b == c)
printf("Triunghi este echilateral: ");
else if(a == b || b == c || a == c)
printf("Triunghi este isoscel: ");
else
printf("Triunghi este scalen ");
return 0; }
```



```
Introdu latura 1: 12
Introdu latura 2: 23
Introdu latura 3: 34
Triunghi este scalen
```

“Figura 17.”

Concluzie

Codul afeseaza care triunghi este in functie de cifre

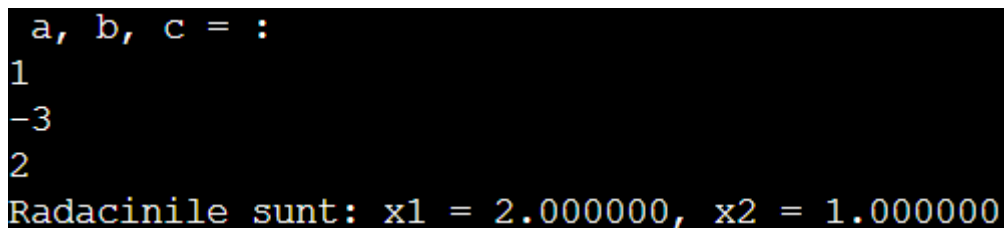
Probleme pentru instrucțiuni de ramificare:

Exercițiu nr.17

17.Scrieți un program C pentru a găsi toate rădăcinile unei ecuații pătratice.

```
#include <stdio.h>
#include <math.h>

int main () {
    double a, b, c, d, x1, x2;
    printf(" a, b, c = :\n");
    scanf("%lf %lf %lf", &a, &b, &c);
    d = b*b - 4*a*c;
    if (d > 0) {
        x1 = (-b + sqrt(d)) / (2*a);
        x2 = (-b - sqrt(d)) / (2*a);
        printf("Radacinile sunt: x1 = %lf, x2 = %lf\n",
x1, x2);
    } else {
        printf("Nu exista radacini reale.\n");}
    return 0;}
```



```
a, b, c = :
1
-3
2
Radacinile sunt: x1 = 2.000000, x2 = 1.000000
```

“Figura 17.”

Concluzia

Programul găsește rădăcinile unei ecuații pătratice. Rădăcinile pot fi reale sau complexe.

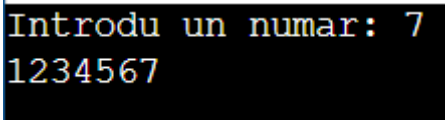
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.1

18.Scrieți un program C pentru a imprima toate numerele naturale de la 1 la n. - folosind bucla while.

```
#include <stdio.h>

int main()
{   int n, i;
    printf("Introdu un numar: ");
    scanf("%d",&n );
    i = 1;
    while(i <= n )
    {
        printf("%d" ,i );
        i++;
    }
    return 0;}
```



```
Introdu un numar: 7
1234567
```

“Figura 18.”

Concluzie

Codul afiseaza ciclu care se incepe de la 1 pana la n.

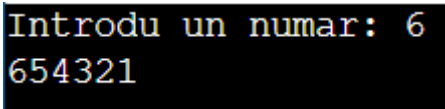
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.2

19.Scrieți un program C pentru a imprima toate numerele naturale invers (de la n la 1). - folosind bucla while.

```
#include <stdio.h>

int main()
{
    int n, i;
    printf("Introdu un numar: ");
    scanf("%d", &n );
    i = n;
    while(i >= 1 )
    {
        printf("%d" ,i );
        i--;
    }
    return 0;}
```



```
Introdu un numar: 6
654321
```

“Figura 19.”

Concluzie

Codul afiseaza ciclu care se incepe de la n pana la 1.

Probleme pentru instrucțiuni ciclice:

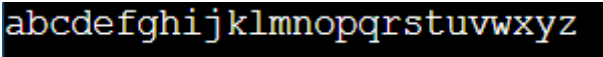
Exercițiu nr.3

20.Scrieți un program C pentru a imprima toate alfabetele de la a la z. - folosind bucla while

```
#include <stdio.h>

int main()
{
    char ch ='a';

    while( ch <= 'z' )
    {
        printf("%c" ,ch );
        ch++;
    }
    return 0;
}
```



abcdefghijklmnopqrstuvwxyz

“Figura 20.”

Concluzie

Codul afiseaza ciclul care se incepe de la a si se termina cu z.

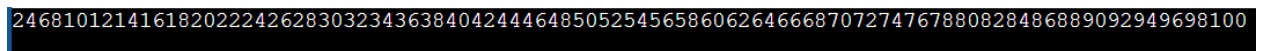
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.4

21.Scrieți un program C pentru a imprima toate numerele pare între 1 și 100. -
utilizând bucla while

```
#include <stdio.h>

int main()
{ int i = 2;
while(i<=100) {
printf("%d", i);
i += 2;}
return 0;}
```



2468101214161820222426283032343638404244464850525456586062646668707274767880828486889092949698100

“Figura 21.”

Concluzie

Programul afișează corect toate numerele pare de la 1 la 100 folosind bucla while.
Variabila crește cu 2 la fiecare pas până la 100.

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.5

23.Scrieți un program C pentru a imprima toate numerele impare între 1 și 100.

```
#include <stdio.h>

int main()
{ int i = 1;
while(i<=100) {
printf("%d ", i);
i += 2;}
return 0;}
```

13579111315171921232527293133353739414345474951535557596163656769717375777981838587899193959799

“Figura 22.”

Concluzie

Programul afișează corect toate numerele impare de la 1 la 99 folosind bucla while.
Variabila crește cu 2 la fiecare pas până la 99

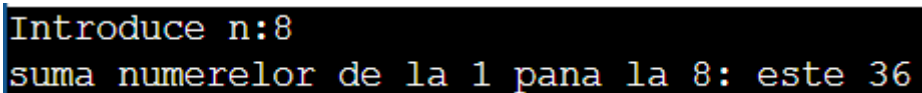
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.6

24. Scrieți un program C pentru a găsi suma tuturor numerelor naturale cuprinse între 1 și n.

```
#include <stdio.h>

int main()
{ int n, suma = 0, i = 1;
printf("Introduce n:");
scanf("%d" , &n);
while(i <= n){
suma += i;
i++;}
printf("suma numerelor de la 1 pana la %d: este
%d\n ", n, suma);
return 0;}
```



```
Introduce n:8
suma numerelor de la 1 pana la 8: este 36
```

“Figura 23.”

Concluzie

Programul calculează corect suma tuturor numerelor naturale de la 1 până la n

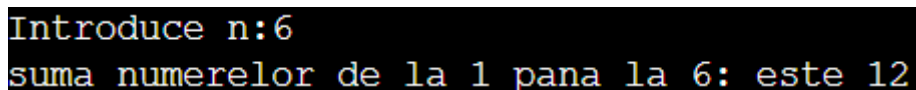
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.7

25.Scrieți un program C pentru a găsi suma tuturor numerelor pare între 1 și n.

```
#include <stdio.h>

int main()
{ int n, suma = 0, i = 2;
printf("Introduce n:");
scanf("%d" ,&n);
while(i <= n){
suma += i;
i +=2 ;}
printf("suma numerelor de la 1 pana la %d: este
%d\n ", n, suma);
return 0;}
```



```
Introduce n:6
suma numerelor de la 1 pana la 6: este 12
```

“Figura 24.”

Concluzie

Programul calculează corect suma tuturor numerelor pare între 1 și n

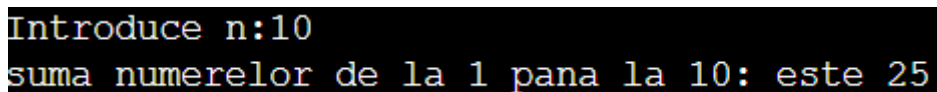
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.8

26.Scrieți un program C pentru a găsi suma tuturor numerelor impare între 1 și n.

```
#include <stdio.h>

int main()
{ int n, suma = 0, i = 1;
printf("Introduce n:");
scanf("%d" , &n);
while(i <= n){
suma += i;
i +=2 ;}
printf("suma numerelor de la 1 pana la %d: este
%d\n ", n, suma);
return 0;}
```



```
Introduce n:10
suma numerelor de la 1 pana la 10: este 25
```

“Figura 24.”

Concluzie

Programul calculează corect suma tuturor numerelor impare între 1 și n

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.9

27.Scrieți un program C pentru a imprima tabelul de înmulțire a oricărui număr.

```
#include <stdio.h>

int main() {
    int n , i=1;

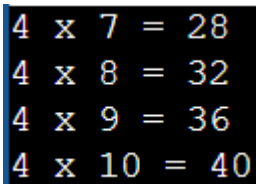
    printf("Introdu numar pentru tabelul de
inmultirie:" );

    scanf("%d", &n);

    printf("tabelul de inmultire pentru %d\n",
n);

    while(i <= 10) {
        printf("%d x %d = %d\n", n, i, n * i);
        i++;
    }

    return 0; }
```



```
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
```

“Figura 25.”

Concluzie

Programul afișează corect tabelul de înmulțire al unui număr dat

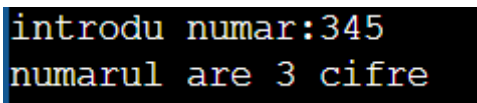
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.10

28.Scrieți un program C pentru a număra numărul de cifre dintr-un număr.

```
#include <stdio.h>

int main()
{ int n,cifre = 0;
printf("introdu numar:");
scanf("%d",&n);
do { n/= 10;
cifre++;}
while(n != 0);
printf("numarul are %d cifre\n",cifre);
return 0;}
```



```
introdu numar:345
numarul are 3 cifre
```

“Figura 26.”

Concluzie

Programul calculează corect numărul de cifre dintr-un număr introdus

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.11

29.Scrieți un program C pentru a găsi prima și ultima cifră a unui număr.

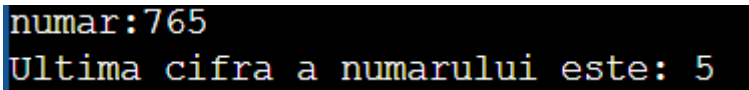
```
#include <stdio.h>

int main() {
    int n;
    printf("numar:");
    scanf("%d", &n);

    int ultimacifra = n % 10;

    printf("Ultima cifra a numarului este: %d\n",
        ultima cifra);

    return 0;
}
```



```
numar:765
Ultima cifra a numarului este: 5
```

“Figura 27.”

Concluzia

Programul afiseaza ultimul numer din numere introduce

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.12

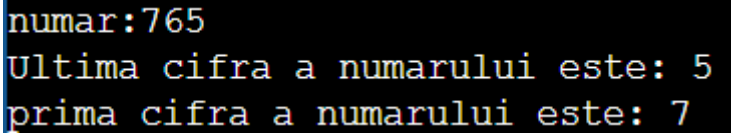
30.Scrieți un program C pentru a găsi suma primei și ultimei cifre a unui număr.

```
#include <stdio.h>

int main() {
    int n;
    printf("numar: " );
    scanf("%d", &n);
    int ultimacifra = n % 10;
    int prima_cifra = n;
    printf("Ultima cifra a numarului este: %d\n",
ultimacifra);

    int primacifra = n;
    while (prima_cifra >= 10) {
        prima_cifra = prima_cifra /10;}
    printf("prima cifra a numarului este: %d\n",
prima_cifra);

    return 0;}
```



```
numar:765
Ultima cifra a numarului este: 5
prima cifra a numarului este: 7
```

“Figura 28”

Concluzie

Codul dat afiseaza ultima si prima cifra

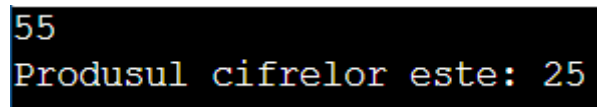
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.15

31.Scrieți un program C pentru a schimba prima și ultima cifră a unui număr.

```
#include <stdio.h>

int main() {
    int n, produs = 1;
    scanf("%d", &n);
    while (n > 0) {
        produs *= n % 10;
        n /= 10;
    }
    printf("Produsul cifrelor este: %d\n", produs);
    return 0;}
```



```
55
Produsul cifrelor este: 25
```

"Figura 29."

Concluzie

Codul afiseaza produsul cifrelor

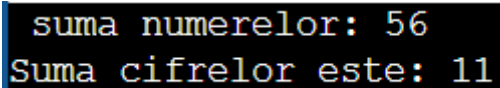
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.14

32.Scrieți un program C pentru a calcula suma cifrelor unui număr.

```
#include <stdio.h>

int main() {
    int suma = 0,n;
    printf(" suma numerelor: ");
    scanf("%d", &n);
    if (n < 0)
        n = -n;
    while (n > 0) {
        suma += n % 10;
        n /= 10;}
    printf("Suma cifrelor este: %d\n", suma);
    return 0;}
```



```
suma numerelor: 56
Suma cifrelor este: 11
```

"Figura 30."

Concluzie

Codul prin operatorul % afla ultima cifra iar cu ajutorul /10 elimina aceasta cifra

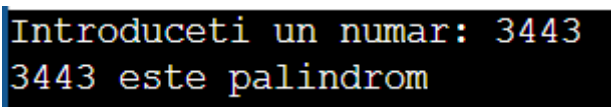
Probleme pentru instrucțiuni ciclice:

Exercițiu nr.17

33.Scrieți un program C pentru a verifica dacă un număr este palindrom sau nu.

```
#include <stdio.h>

int main() {
    int n, original, invers=0, cifra;
    printf("Introduceti un numar: ");
    scanf("%d", &n);
    original = n;
    while (n > 0) {
        cifra = n % 10;
        invers = invers * 10 + cifra;
        n /= 10;
    }
    if (original == invers) {
        printf("%d este palindrom\n", original);
    } else {
        printf("%d nu este palindrom\n", original);}
    return 0;}
```



```
Introduceti un numar: 3443
3443 este palindrom
```

“Figura 31.”

Concluzie

Programul verifică dacă un număr este palindrom comparând numărul cu varianta lui inversată.

Probleme pentru instrucțiuni ciclice:

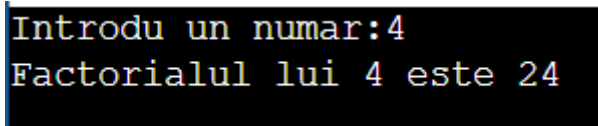
Exercițiu nr.23

34.Scrieți un program C pentru a calcula factorialul unui număr.

```
#include<stdio.h>

int main()
{
    int n, f = 1;
    printf("Introdu un numar:");
    scanf("%d",&n);
    for (int i = 1; i <= n; i++){
        f = f * i;
    }

    printf("Factorialul lui %d este %d\n", n, f);
    return 0;
}
```



```
Introdu un numar:4
Factorialul lui 4 este 24
```

“Figura 32.”

Concluzie

Programul calculează factorialul unui număr introdus, folosind o buclă for și afișează rezultatul final.

Probleme pentru instrucțiuni ciclice:

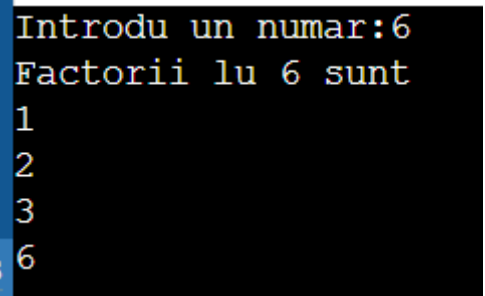
Exercițiu nr.22

35.Scrieți un program C pentru a găsi toți factorii unui număr.

```
#include<stdio.h>
int main()
{
    int n = 1;
    printf("Introdu un numar:");
    scanf("%d",&n);

    printf("Factorii lu %d sunt \n",n);
    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {

            printf("%d\n", i);
        }
    }
    return 0;
}
```



```
Introdu un numar:6
Factorii lu 6 sunt
1
2
3
6
```

“Figura 33.”

Concluzie

Programul găsește și afișează toți factorii unui număr introdus de utilizator.

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.45

36.Scrieți un program C pentru a converti sistemul numeric Zecimal în Binar.

```
#include<stdio.h>
int main() {
    int n;

    printf("Introdu un numar: ");

    scanf("%d", &n);

    printf("Numarul binar este: ");
    for (int i = 31; i >= 0; i--) {
        printf("%d", (n >> i) & 1);
    }
    return 0;
}
```

```
Introdu un numar: 4  
Numarul binar este: 0000000000000000000000000000100
```

“Figura 34.”

Concluzie

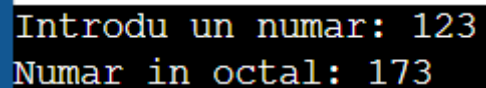
Programul transformă un număr zecimal introdus de utilizator în binar

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.43

35.Scrieți un program C pentru a converti sistemul numeric Octal în Decimal.

```
#include<stdio.h>
int main() {
    int n;
    printf("Introdu un numar: ");
    scanf("%d", &n);
    printf("Numar in octal: %o\n",n);
    return 0; }
```



```
Introdu un numar: 123
Numar in octal: 173
```

“Figura 35.”

Concluzie

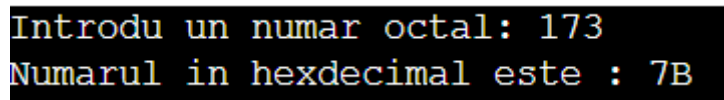
Programul transformă un număr zecimal introdus de utilizator în octal

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.44

36.Scrieți un program C pentru a converti sistemul de numere octal în hexazecimal

```
#include<stdio.h>
int main() {
    int n;
    printf("Introdu un numar octal: ");
    scanf("%o",&n);
    printf("Numarul in hexadecimal este : %X",n);
    return 0;}
```



```
Introdu un numar octal: 173
Numarul in hexadecimal este : 7B
```

"Figura 36."

Concluzie

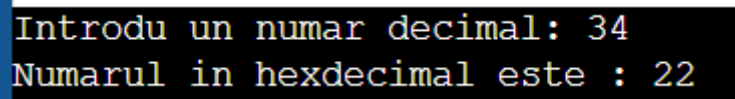
Programul citește un număr octal și îl afișează direct în hexazecimal

Probleme pentru instrucțiuni ciclice:

Exercițiu nr.47

37.Scrieți un program C pentru a converti sistemul numeric Decimal în Hexadecimal.

```
#include<stdio.h>
int main() {
    int n;
    printf("Introdu un numar decimal: ");
    scanf("%d",&n);
    printf("Numarul in hexadecimal este : %X",n);
    return 0;}
```



```
Introdu un numar decimal: 34
Numarul in hexadecimal este : 22
```

“Figura 37.”

Concluzia

Programul citește un număr decimal și îl afișează direct în hexadecimal

Concluzie

La finele lucrării studentul trebuie să cunoască:

- ce instrucțiuni de ramificare sunt în limbajul C?
- ce instrucțiuni ciclice sunt în limbajul C?
- care este diferența între while și do while?
- care este diferența între for și while?
- care este diferența între if și switch?
- ce înseamnă break, continue ?
- ce înseamnă contor, condiție de oprire și pasul contorului?